SISTEMAS DISTRIBUIDOS

LABORATORIO S11

Dos bases más y una caché

Fecha de entrega: 06 de noviembre, 2026
Profesor: Ing. Oscar Manual Ruiz Hurtado

Alumno: Echeverría Goicochea Erick Isaac
Equipo: DiDistribuidores

No. Cuenta: 424153520
Grupo: 2
Semestre: 2027-1

Bitácora Laboratorio S11 · Mexi Banco: una verdad, varias copias

Alumno: Echeverria Goicochea Erick Isaac

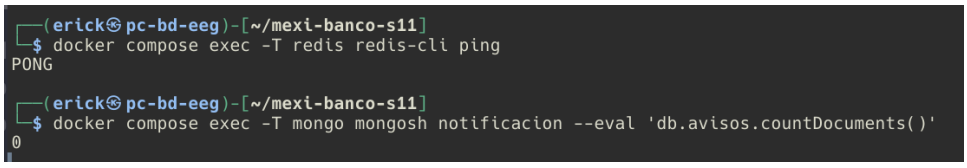
1. El punto de partida

Qué hice. Levanté el banco con docker compose y revisé que todos los contenedores estuvieran healthy, incluidos redis, mongo y las tres copias de cuenta (Ilustración 1). Comprobé que Redis contesta PONG y que en Mongo todavía no había avisos (Ilustración 2). Después abrí las cuentas de Ana (1000), Beto (500) y Carla (300): Ana y Beto ya existían de otro laboratorio, por eso dieron 409, y Carla dio 201 (Ilustración 3). Hice una transferencia de 200 de Ana a Beto, que respondió COMPLETADA (Ilustración 4).



```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose ps
NAME                              IMAGE                          COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-cuenta-1           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-2           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-cuenta-3           mexi-banco-v06-cuenta          "java -jar app.jar"      cuenta          55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-db-1               postgres:17-alpine             "docker-entrypoint.s…"   db              8 hours ago      Up 54 minutes (healthy)   5432/tcp
mexi-banco-v06-discovery-1        mexi-banco-v06-discovery       "java -jar app.jar"      discovery       55 minutes ago   Up 54 minutes (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-gateway-1          mexi-banco-v06-gateway         "java -jar app.jar"      gateway         55 minutes ago   Up 54 minutes (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-v06-mongo-1            mongo:8.0                      "docker-entrypoint.s…"   mongo           8 hours ago      Up 54 minutes (healthy)   127.0.0.1:27017->27017/tcp
mexi-banco-v06-movimiento-1       mexi-banco-v06-movimiento      "java -jar app.jar"      movimiento      55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-notificacion-1     mexi-banco-v06-notificacion    "java -jar app.jar"      notificacion    55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-rabbitmq-1         rabbitmq:4-management          "docker-entrypoint.s…"   rabbitmq        2 weeks ago      Up 54 minutes (healthy)   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5
>5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
mexi-banco-v06-redis-1            redis:8-alpine                 "docker-entrypoint.s…"   redis           8 hours ago      Up 54 minutes (healthy)   127.0.0.1:6379->6379/tcp
mexi-banco-v06-spei-1             mexi-banco-v06-spei            "java -jar app.jar"      spei            55 minutes ago   Up 54 minutes (healthy)   8080/tcp
mexi-banco-v06-transferencia-1    mexi-banco-v06-transferencia   "java -jar app.jar"      transferencia   55 minutes ago   Up 54 minutes (healthy)   8080/tcp
```

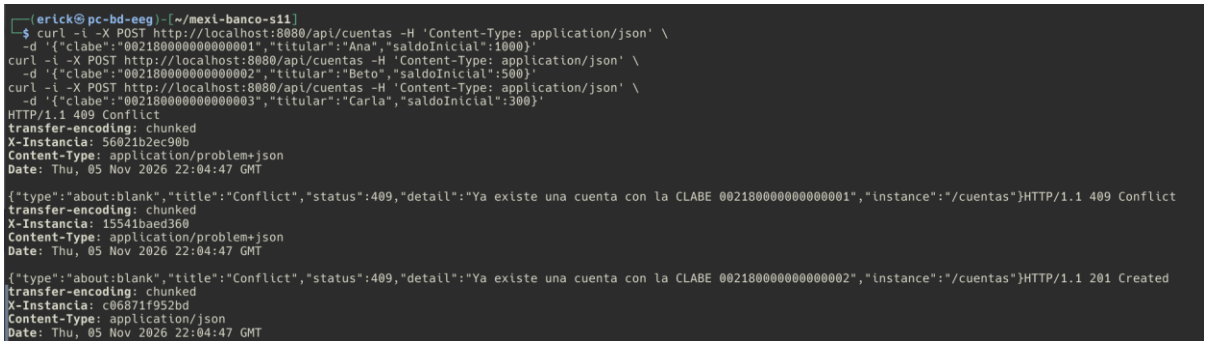
Ilustración 1. docker compose ps con todos los contenedores healthy



```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T redis redis-cli ping
PONG

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
0
```

Ilustración 2. Redis contesta PONG y la colección de avisos en Mongo tiene 0 documentos.



```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000001","titular":"Ana","saldoInicial":1000}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000002","titular":"Beto","saldoInicial":500}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"002180000000000003","titular":"Carla","saldoInicial":300}'
HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 56021b2ec90b
Content-Type: application/problem+json
Date: Thu, 05 Nov 2026 22:04:47 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000001","instance":"/cuentas"}HTTP/1.1 409 Conflict
transfer-encoding: chunked
X-Instancia: 15541baed360
Content-Type: application/problem+json
Date: Thu, 05 Nov 2026 22:04:47 GMT

{"type":"about:blank","title":"Conflict","status":409,"detail":"Ya existe una cuenta con la CLABE 002180000000000002","instance":"/cuentas"}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: c06871f952bd
Content-Type: application/json
Date: Thu, 05 Nov 2026 22:04:47 GMT
```

Ilustración 3. Alta de las cuentas: Ana y Beto ya existían (409) y Carla se creó (201).



```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200}'
{"id":27,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200,"fecha":"2026-11-06T05:24:14.498929135Z","estado":"COMPLETADA",
"motivo":null,"saldoOrigenTrasCargo":800.00,"intentosDeCompensacion":0,"actualizada":"2026-11-06T05:24:14.661989668Z","eventoId":"7a4a9079-ca6a-4667-8b95-0
4b998a18fa0"}
```

Ilustración 4. Transferencia de 200 de Ana a Beto: COMPLETADA.

Mi "antes". Pedí el saldo de Ana diez veces seguidas. Las diez respuestas traen un leidaEn distinto, es decir, que las diez se leyeron de Postgres. El log tiene la línea "leida de Postgres" 24 veces: son las diez consultas más las lecturas de la transferencia y de las pruebas anteriores. En Redis, keyspace_hits y keyspace_misses están en 0 porque nadie lo usaba todavía.

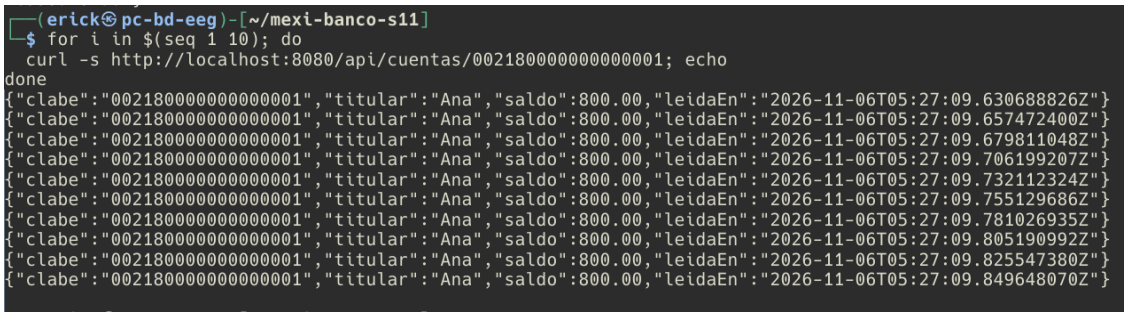


Ilustración 5. Diez consultas del saldo de Ana: cada una con un leidaEn distinto.



Ilustración 6. Conteo de lecturas a Postgres en el log y contadores de Redis en 0.

2. La caché del saldo en Redis

Usé el patrón cache-aside: la app pregunta primero a Redis; si la copia no está (miss), lee Postgres y deja una copia para la próxima vez; si está (hit), contesta Redis y Postgres ni se entera. Cuando el saldo cambia, la copia se borra y la siguiente consulta la vuelve a llenar. Esto es lo que cambié en el servicio cuenta:

1. **ConfiguracionCache.** Agregué @EnableCaching y configuré la caché cuentas con tres cosas: un TTL de 30 segundos para que ninguna copia viva para siempre, el valor guardado como JSON de CuentaVista para poder leerlo con redis-cli, y transactionAware() para que el borrado espere al commit.
2. **ConsultaDeSaldo.** Puse @Cacheable en el método consultar, con la clave "#clabe", que en Redis queda como cuentas::<clabe>. Es la consulta que solo muestra el saldo.
3. **CuentaService.** Puse @CacheEvict con la misma clave en cargar y en abonar, junto al @Transactional que ya tenían. No toqué el consultar de CuentaService: es el que usa el cargo para decidir si alcanza el

Predicción. Antes de correr la prueba predije que, al pedir seis veces el saldo de Ana, Postgres se leería una sola vez: la primera consulta sería un miss y las otras cinco serían hits, aunque las contestaran copias distintas de cuenta, porque Redis es uno solo y lo comparten las tres.

Resultado. Pedí el saldo de Ana seis veces. X-Instancia cambia entre las tres copias, pero leidaEn es siempre el mismo. El log registró una sola lectura a Postgres, y en Redis la copia aparece como JSON con un TTL de 29 segundos. Así comprobé que las tres copias comparten la caché.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'
done

docker compose logs --since 30s cuenta | grep -c "leida de Postgres"
docker compose exec -T redis redis-cli GET cuentas::002180000000000001
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
X-Instancia: 8e354edbcfce
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
X-Instancia: 1dfc2852c750
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
X-Instancia: 47bcdbd23109
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
X-Instancia: 8e354edbcfce
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
X-Instancia: 1dfc2852c750
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
X-Instancia: 47bcdbd23109
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
1
{"clabe":"002180000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-06T05:47:25.007783925Z"}
29
```

Ilustración 7. Seis consultas contestadas por tres copias distintas con el mismo leidaEn, una sola lectura a Postgres.

La copia se borra al transferir. Con redis MONITOR abierto, transferí 10 de Ana a Beto. Después de la transferencia, EXISTS da 0 (la copia de Ana ya no está), y al volver a consultar el saldo sale el valor nuevo (790) con un leidaEn nuevo. En el MONITOR se ve el orden: el GET de la consulta, el SET con "PX 30000" (la copia vive 30 s), los DEL de la cuenta de Ana y de la de Beto después del commit, y luego el GET (miss) y el SET con el saldo nuevo.



Ilustración 8. Transferencia de 10, EXISTS en 0 y el saldo nuevo (790.00) con leidaEn nuevo.

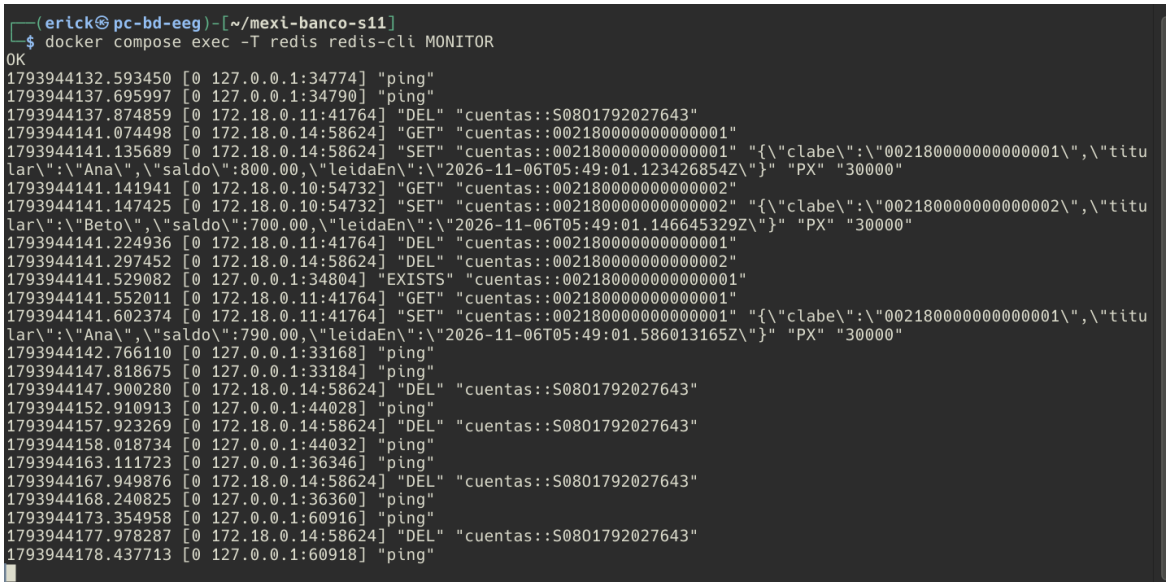


Ilustración 9. MONITOR de Redis: SET con PX 30000, los DEL después del commit y el nuevo SET con el saldo 790.

3. Mis medidas

Hit ratio. El hit ratio es hits entre (hits + misses). En el escenario A pedí cien veces el saldo de Ana: 99 hits y 1 miss, o sea 99 %. En el escenario B hice lo mismo pero con una transferencia de 1 peso cada diez

consultas: 101 hits y 19 misses, o sea 84 %. Baja porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada uno falla.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
keyspace_hits:99
keyspace_misses:1

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"00218000000000001","claveDestino":"002180000000000002","monto":1}'
  fi
done
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
keyspace_hits:101
keyspace_misses:19
```

Ilustración 10. Escenario A (99 hits, 1 miss) y escenario B (101 hits, 19 misses).

Latencia. Medí cien consultas con caché dos veces y cien consultas sin caché, apagándola con CACHE_TIPO=none.

Caso	Promedio	Mediana	p95	Lecturas a Postgres
Con caché, corrida 1	7.15 ms	6.20 ms	8.14 ms	3
Con caché, corrida 2	7.80 ms	7.27 ms	8.45 ms	2
Sin caché	11.33 ms	10.39 ms	11.89 ms	100

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ for i in $(seq 1 100); do   curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuentas/002180000000000001; done | sort -n
  | awk '{t[NR]=$1*1000; s+=$1*1000}
       END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=7.15 ms mediana=6.20 ms p95=8.14 ms

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose logs --since 2m cuenta | grep -c "leida de Postgres"
3
```

Ilustración 11. Con caché, corrida 1: promedio 7.15 ms, mediana 6.20 ms, p95 8.14 ms y 3 lecturas a Postgres.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuentas/002180000000000001
done | sort -n \
  | awk '{t[NR]=$1*1000; s+=$1*1000}
         END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=7.80 ms mediana=7.27 ms p95=8.45 ms

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose logs --since 2m cuenta | grep -c "leida de Postgres"
2
```

Ilustración 12. Con caché, corrida 2: promedio 7.80 ms, mediana 7.27 ms, p95 8.45 ms y 2 lecturas a Postgres.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ CACHE_TIPO=none docker compose up -d cuenta
[+] Running 6/6
 ✔ Container mexi-banco-v06-db-1         Healthy
 ✔ Container mexi-banco-v06-discovery-1  Healthy
 ✔ Container mexi-banco-v06-redis-1      Healthy
 ✔ Container mexi-banco-v06-cuenta-1     Started
 ✔ Container mexi-banco-v06-cuenta-3     Started
 ✔ Container mexi-banco-v06-cuenta-2     Started

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ for i in $(seq 1 100); do
  curl -s -o /dev/null -w '%{time_total}\n' http://localhost:8080/api/cuentas/002180000000000001
done \
  | sort -n \
  | awk '{t[NR]=$1*1000; s+=$1*1000}
         END {printf "promedio=%.2f ms mediana=%.2f ms p95=%.2f ms\n", s/NR, t[int(NR*0.5)], t[int(NR*0.95)]}'
promedio=11.33 ms mediana=10.39 ms p95=11.89 ms

┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose logs --since 2m cuenta | grep -c "leida de Postgres"
100
```

Ilustración 13. Sin caché: promedio 11.33 ms, mediana 10.39 ms, p95 11.89 ms y 100 lecturas a Postgres.

Qué compré con la caché. La consulta fue algo más rápida (mediana de 6.2 y 7.3 ms contra 10.4 ms sin caché), pero lo más importante es la capacidad: con caché Postgres recibió 3 y 2 lecturas por cada 100 consultas, y sin caché recibió las 100. La caché deja libre a Postgres para los cargos y los abonos.

4. Los avisos en MongoDB

Cada aviso es un documento con los campos que necesita su canal: SMS con teléfono y texto, CORREO con correo, asunto y cuerpo, y PUSH con token, título y cuerpo. Nadie tiene que hacer ALTER TABLE para que existan, y no hay columnas vacías. Esto es lo que programé en BitacoraDeAvisos:

1. **documento(...).** Arma el aviso como un Document. El _id es eventoId:clabe (un evento genera dos avisos, uno por cada CLABE), el monto y el saldo van como Decimal128 y la fecha es la del evento. Un switch sobre Contacto agrega los campos de cada canal, como Contacto es una interfaz sellada, si faltara un canal no compilaría.
2. **registrar(evento).** Inserta los avisos uno por uno. Si Mongo rechaza el _id porque ya existe (DuplicateKeyException), ese aviso no se cuenta y se sigue con el siguiente: no es un error, es un evento repetido. Devuelve cuántos avisos eran nuevos.
3. **crearIndice() y ultimos(clabe).** El índice clabe_fecha (clabe ascendente, fecha descendente) sigue la forma de la consulta de los últimos 10: filtra por CLABE y ordena por fecha. ultimos busca por CLABE, ordena por fecha descendente y se queda con 10.
4. **El consumidor.** En ConsumidorDeTransferencias cambié una línea: ahora hace nuevos = bitacora.registrar(evento). Los avisos van solo a Mongo y ya no a Postgres, porque escribir en los dos sería una doble escritura sin transacción común.

Prueba. Al reconstruir notificación, el log dice que el índice clabe_fecha quedó listo. Hice una transferencia de 200 de Ana a Beto y otra de 200 de Ana a Carla (Ilustración 15). En mongosh se ven las tres formas: el SMS de Ana y el CORREO de Beto (Ilustración 16) y el PUSH de Carla con su token (Ilustración 17). Un aviso no tiene los campos de otro canal. Al filtrar por los que tienen teléfono, solo salen los SMS de Ana (Ilustración 18).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-06T06:24:51.829Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos
      : [avisos] indice clabe_fecha listo en avisos
```

Ilustración 14. El log de notificación: índice clabe_fecha listo en avisos.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200}'
{"id":39,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200,"fecha":"2026-11-06T06:31:26.867631036Z","estado
":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":580.00,"intentosDeCompensacion":0,"actualizada":"2026-11-06T06:31:27.088841501Z","evento
Id":"d6d3f7ad-f09d-44dc-8852-bf06db99a268"}
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json'   -d '{"claveOrigen":"00218000000000001","
claveDestino":"00218000000000003","monto":200}'
{"id":40,"claveOrigen":"00218000000000001","claveDestino":"00218000000000003","monto":200,"fecha":"2026-11-06T06:31:51.836107259Z","estado
":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":380.00,"intentosDeCompensacion":0,"actualizada":"2026-11-06T06:31:51.962592432Z","evento
Id":"1cafef3d-dae9-48eb-931d-16c9a16804a0"}
```

Ilustración 15. Transferencias de 200 de Ana a Beto y de Ana a Carla.

```
    _id: '2d0b4ef8-7b99-4df9-ae4e-270beb5cd4e0:002180000000000001',
    eventoId: '2d0b4ef8-7b99-4df9-ae4e-270beb5cd4e0',
    transferenciaId: Long('36'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('1'),
    saldo: Decimal128('782.00'),
    fecha: ISODate('2026-11-06T05:54:07.583Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 1, saldo resultante 782.00'
  },
  {
    _id: '2d0b4ef8-7b99-4df9-ae4e-270beb5cd4e0:002180000000000002',
    eventoId: '2d0b4ef8-7b99-4df9-ae4e-270beb5cd4e0',
    transferenciaId: Long('36'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('1'),
    saldo: Decimal128('718.00'),
    fecha: ISODate('2026-11-06T05:54:07.583Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 1, saldo resultante 718.00'
  }
]
Type "it" for more
```

Ilustración 16. Ejemplo de un SMS (Ana) y un CORREO (Beto) en la colección avisos.

```
{
  _id: '1cafef3d-dae9-48eb-931d-16c9a16804a0:002180000000000003',
  eventoId: '1cafef3d-dae9-48eb-931d-16c9a16804a0',
  transferenciaId: Long('40'),
  clabe: '002180000000000003',
  tipo: 'TRANSFERENCIA_RECIBIDA',
  monto: Decimal128('200'),
  saldo: Decimal128('500.00'),
  fecha: ISODate('2026-11-06T06:31:51.962Z'),
  canal: 'PUSH',
  token: 'fcm:demo-002180000000000003',
  titulo: 'Recibiste una transferencia',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 500.00'
}
```

Ilustración 17. Ejemplo de un PUSH (Carla), con token y título.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ # Muestra solo los que tienen teléfono, con tres campos: el esquema lo aplica quien lee
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '7a4a9079-ca6a-4667-8b95-04b998a18fa0:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '22f09e8a-51fc-449b-b789-d481be8bea5c:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
```

Ilustración 18. Filtro por los avisos que tienen teléfono: solo los SMS de Ana.

Los últimos 10 y el índice. Hice doce transferencias de 1 peso de Ana a Beto y pedí la bitácora de Beto por el gateway: devuelve solo los 10 avisos más recientes. getIndexes() muestra el índice clabe_fecha además de _id_ (Ilustración 20).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ for i in $(seq 1 12); do curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'; done
curl -s http://localhost:8080/api/notificaciones/002180000000000002
[{"_id":"e10062b1-496e-4603-bddf-8a5c93e1214b:002180000000000002","eventoId":"e10062b1-496e-4603-bddf-8a5c93e1214b","transferenciaId":48,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":928.00,"fecha":"2026-11-06T06:45:18.271+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 928.00"}
,{"_id":"80ade1bf-f10f-47ea-9c94-34ef85d2424b:002180000000000002","eventoId":"80ade1bf-f10f-47ea-9c94-34ef85d2424b","transferenciaId":47,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":927.00,"fecha":"2026-11-06T06:45:18.193+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 927.00"}
,{"_id":"538d831c-87ad-4101-8a4e-9a7db698e095:002180000000000002","eventoId":"538d831c-87ad-4101-8a4e-9a7db698e095","transferenciaId":46,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":926.00,"fecha":"2026-11-06T06:45:18.114+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 926.00"}
,{"_id":"275147aa-f8f2-409e-87f9-132558f9f1eb:002180000000000002","eventoId":"275147aa-f8f2-409e-87f9-132558f9f1eb","transferenciaId":45,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":925.00,"fecha":"2026-11-06T06:45:18.038+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 925.00"}
,{"_id":"ee6a39ac-3eec-489e-8f18-cdca1e668404:002180000000000002","eventoId":"ee6a39ac-3eec-489e-8f18-cdca1e668404","transferenciaId":44,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":924.00,"fecha":"2026-11-06T06:45:17.960+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 924.00"}
,{"_id":"5f4b1eda-c470-49b4-8d22-28ed71dc950f:002180000000000002","eventoId":"5f4b1eda-c470-49b4-8d22-28ed71dc950f","transferenciaId":43,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":923.00,"fecha":"2026-11-06T06:45:17.874+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 923.00"}
,{"_id":"0ca71d6e-0f6f-4490-ac4a-9595c858a152:002180000000000002","eventoId":"0ca71d6e-0f6f-4490-ac4a-9595c858a152","transferenciaId":42,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":922.00,"fecha":"2026-11-06T06:45:17.785+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 922.00"}
,{"_id":"a37e7f34-aefa-41cf-aac1-294e23559664:002180000000000002","eventoId":"a37e7f34-aefa-41cf-aac1-294e23559664","transferenciaId":41,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:1,"saldo":921.00,"fecha":"2026-11-06T06:45:17.700+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 921.00"}
,{"_id":"d6d3f7ad-f09d-44dc-8852-bf06db99a268:002180000000000002","eventoId":"d6d3f7ad-f09d-44dc-8852-bf06db99a268","transferenciaId":39,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto"
:200,"saldo":920.00,"fecha":"2026-11-06T06:31:27.888+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 200, saldo resultante 920.
00"},{"_id":"ba01793c-7e7a-48b5-846f-2826cac8cb40:002180000000000002","eventoId":"ba01793c-7e7a-48b5-846f-2826cac8cb40","transferenciaId":38,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","mo
nto":1,"saldo":720.00,"fecha":"2026-11-06T05:54:07.973+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 720.
00"}]
```

Ilustración 19. Doce transferencias de 1 peso y la bitácora de Beto por el gateway (los últimos 10 avisos).

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ # Lista los índices; debe aparecer clabe_fecha además de _id_
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]
```

Ilustración 20. getIndexes(): _id_ y clabe_fecha ({ clabe: 1, fecha: -1 }).

Con índice y sin índice. Pedí a Mongo el plan de la consulta de ultimos() dos veces. Con el índice usa IXSCAN (LIMIT, FETCH, IXSCAN) y examina 10 claves y 10 documentos. Con hint({$natural: 1}) obligué a Mongo a no usarlo: hace COLLSCAN, examina los 52 documentos de la colección y los ordena en memoria (SORT). Los milisegundos casi no cambian porque la colección es chica; lo que importa es cuántos documentos revisa. Con millones de avisos, uno leería 10 y el otro todos.

	Etapas	**Documentos examinados**
Con índice	LIMIT ← FETCH ← IXSCAN	10
Sin índice	SORT ← COLLSCAN	52

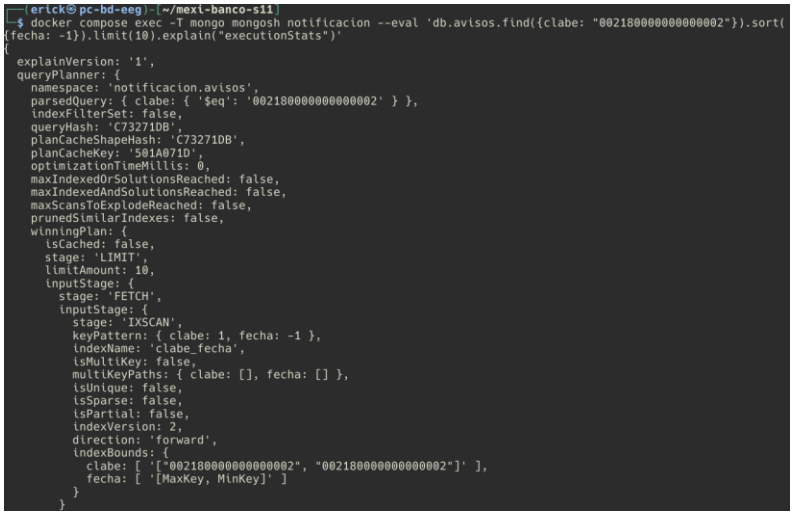


Ilustración 21. explain con índice: etapa IXSCAN con el índice clabe_fecha.

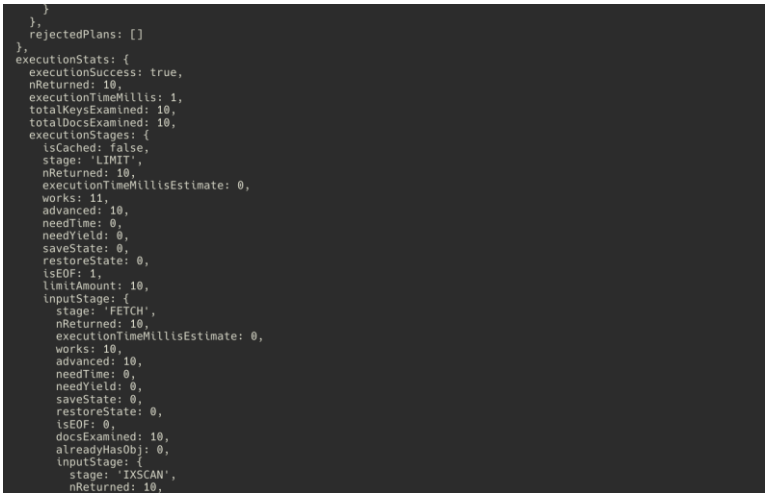


Ilustración 22. explain con índice: totalKeysExamined 10 y totalDocsExamined 10.

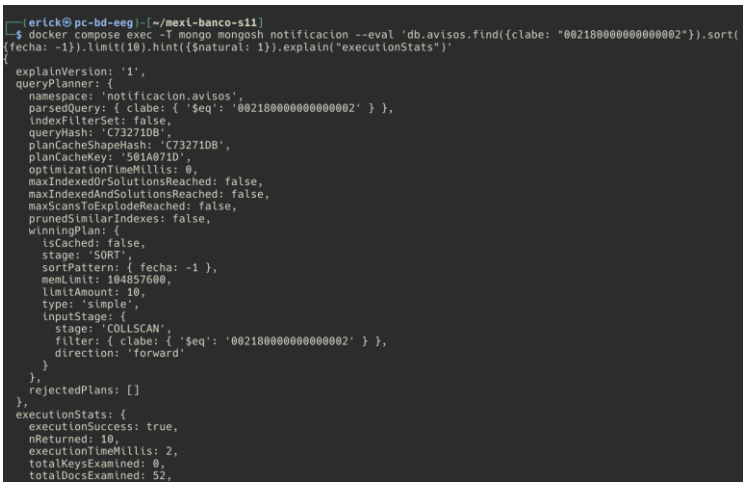


Ilustración 23. explain sin índice (hint $natural)

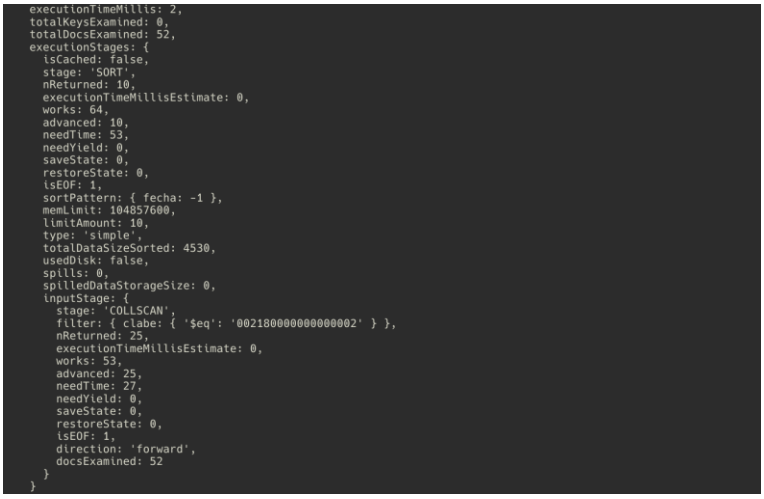


Ilustración 24. explain sin índice: totalDocsExamined 52.

5. El evento duplicado

La mensajería de la S09 entrega al menos una vez, así que un evento puede llegar dos veces. Lo provoqué publicando el mismo evento dos veces desde la API del panel de RabbitMQ, con el mismo eventoId (11111111-…) y CLABEs inventadas (091 y 092). Las dos veces respondió {"routed":true}.



Ilustración 25. Primer publish del evento: {"routed":true}.

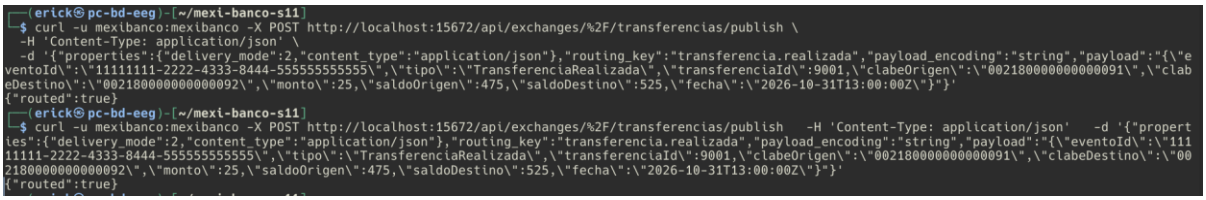


Ilustración 26. Segundo publish, idéntico al primero: {"routed":true}.

Predicción. Dos eventos iguales generan cuatro avisos si nada lo evita, pero predije que quedarían solo dos: el segundo evento arma los mismos dos _id y Mongo debería rechazar los dos inserts.

Resultado. countDocuments da 2, no 4. En el log salen dos líneas "ya estaba: evento repetido, no se duplica", una por cada aviso rechazado. El consumidor muestra dos entregas del mismo evento: la primera con "2 aviso(s) nuevo(s)" y la segunda con "0 aviso(s) nuevo(s)". El _id eventoId:clabe hace de llave única, sin necesidad de una tabla aparte.



Ilustración 27. countDocuments del evento duplicado: 2 documentos.



Ilustración 28. Log de notificación: los dos avisos rechazados ("ya estaba").

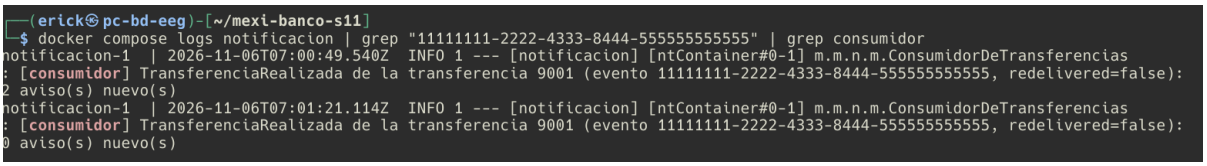


Ilustración 29. Las dos entregas del consumidor: 2 aviso(s) nuevo(s) y después 0 aviso(s) nuevo(s).

6. El dato viejo

La caché puede mentir. Lo probé cambiando el saldo de Ana en Postgres por fuera de la app, con un UPDATE a mano. Así no pasa por @CacheEvict y nadie borra la copia de Redis.

Predicción. Predije que el cliente seguiría viendo el saldo viejo hasta que la copia caducara, unos 30 segundos, y que una transferencia de 100 no pasaría porque el cargo lee Postgres, no la caché.

Resultado. Ana tenía 367. Después de ponerle 5 en Postgres, las primeras vueltas siguen mostrando 367.00 mientras el TTL baja (29, 24, 19, 14, 9, 3). Cuando el TTL llega a -2 la copia caducó, se lee Postgres y aparece 5.00 con un leidaEn nuevo. La copia se guardó a las 07:21:25 y la lectura nueva fue a las 07:21:57: el cliente vio un saldo que ya no existía durante unos 32 segundos.

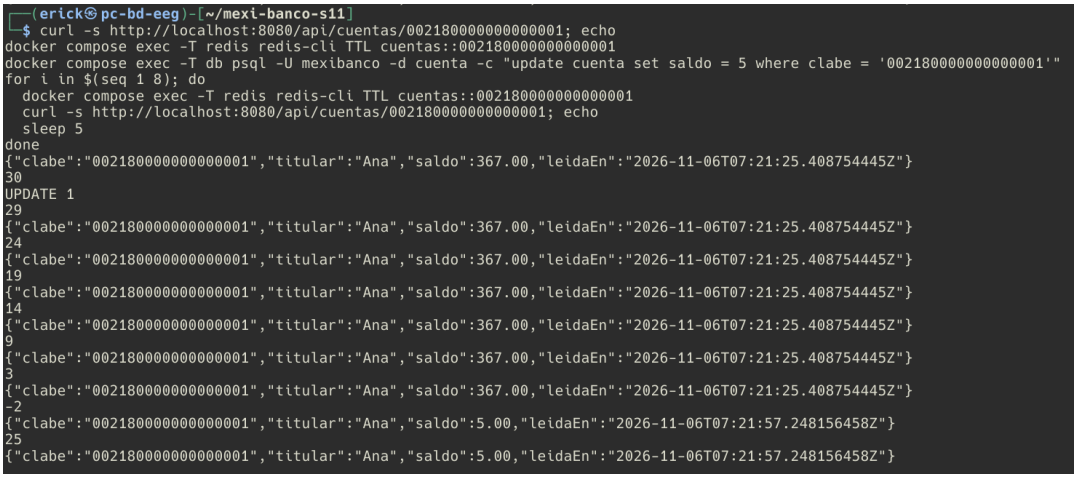


Ilustración 30. El ciclo de 8 vueltas: saldo viejo (367.00) mientras baja el TTL y salto a 5.00 al caducar.

La transferencia de 100. Mientras la pantalla todavía mostraba 367, intenté transferir 100 de Ana a Beto. Respondió 422, "Saldo insuficiente". El cargo autoriza con la lectura que va a Postgres, donde Ana tiene 5, así que la caché engañó a la pantalla pero no a la decisión. Es la regla 2 de hoy: la caché muestra, nunca decide.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":100}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 0021800000
00000001","instance":"/transferencias","servicio":"cuenta"}
```

Ilustración 31. Transferencia de 100 con el saldo viejo en la caché: 422 Saldo insuficiente.

Dejé todo como estaba. Regresé el saldo de Ana a 367 y borré su copia de Redis, porque el UPDATE rompió a propósito el cuadré entre el saldo y los movimientos.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 367 where clabe = '00218000000000001'"
docker compose exec -T redis redis-cli DEL cuentas::00218000000000001
curl -s http://localhost:8080/api/cuentas/00218000000000001
UPDATE 1
0
{"clabe":"00218000000000001","titular":"Ana","saldo":367.00,"leidaEn":"2026-11-06T07:24:09.729621815Z"}
```

Ilustración 32. Saldo de Ana restaurado a 367 y copia de Redis borrada.

7. Redis se cae (reto)

La caché es solo una copia para mostrar, así que no debería poder tumbar nada. Primero comprobé cómo estaba mi versión sin el reto: con Redis encendido, Ana tenía 367 y Beto 933 en Postgres.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose exec -T db psql -U mexibanco -d cuenta -c "select clabe, saldo from cuenta where clabe in ('00218000000000001','
00218000000000002')"
       clabe        | saldo
--------------------+--------
 00218000000000002 | 933.00
 00218000000000001 | 367.00
(2 rows)
```

Ilustración 33. Saldos reales en Postgres antes de apagar Redis: Ana 367 y Beto 933.

El "antes". Apagué Redis (docker compose stop redis) y probé un GET del saldo y una transferencia de 1 peso. Las dos respondieron 500 Internal Server Error: el GET falló porque la caché no pudo conectar, y la transferencia falló porque la saga consulta las cuentas por ese mismo GET. Revisé Postgres y los saldos seguían igual (Ana 367, Beto 933): ese fallo no movió dinero, pero una caché que solo debía acelerar acabó tumbando el servicio.

```
┌──(erick㉿pc-bd-eeg)-[~/mexi-banco-s11]
└─$ docker compose stop redis
curl -i http://localhost:8080/api/cuentas/00218000000000001
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":1}'
[+] Stopping 1/1
 ✔ Container mexi-banco-v06-redis-1  Stopped
      0.5s
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
X-Instancia: f91237c3a849
Content-Type: application/json
Date: Fri, 06 Nov 2026 07:35:43 GMT

{"timestamp":"2026-11-06T07:35:44.408+00:00","status":500,"error":"Internal Server Error","path":"/cuentas/002180000000000
001"}HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/problem+json
Date: Fri, 06 Nov 2026 07:35:44 GMT

{"type":"about:blank","title":"Internal Server Error","status":500,"detail":"El servicio cuenta respondio 500","instance":
"/transferencias","servicio":"cuenta"}
```

Ilustración 34. Con Redis apagado y mi versión anterior: el GET y la transferencia responden 500.

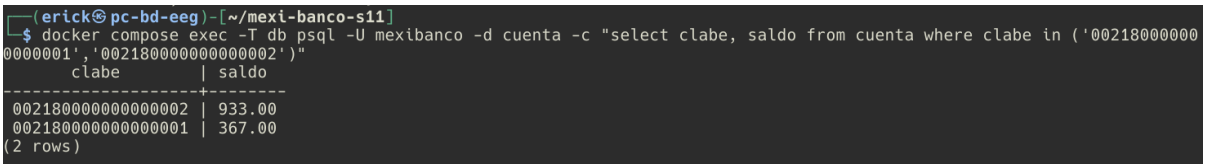


Ilustración 35. Saldos después del intento fallido: Ana 367 y Beto 933, no se movió dinero.

Por qué no basta un CacheErrorHandler. Spring trae CacheErrorHandler para atrapar errores de la caché, pero el DEL de @CacheEvict, con transactionAware(), no corre dentro del interceptor: se ejecuta después del commit. Si Redis está caído y ese DEL lanza una excepción, sale cuando el cargo ya se confirmó en Postgres, y cuenta contesta 500 a un cargo que ya quedó guardado. La saga creería que el cargo falló, aunque el dinero ya salió. Por eso hay que atrapar el error más abajo, en la pieza que de verdad habla con Redis.

El arreglo. Creé CacheWriterTolerante, que envuelve al RedisCacheWriter (el builder de ConfiguracionCache acepta .cacheWriter(...)). Si Redis falla: un get devuelve null (para Spring es un miss, así que corre el método y lee Postgres), un put se ignora y un remove, que es el DEL de después del commit, también se ignora. En todos los casos deja un WARN con la operación y la clave que fallaron. Es un proxy del interfaz, así no depende de qué métodos tenga RedisCacheWriter en cada versión.

```
return builder -> builder

.cacheWriter(CacheWriterTolerante.de(RedisCacheWriter.nonLockingRedisCacheWriter(conexiones)))
    .withCacheConfiguration(CUENTAS, cuentas)
    .transactionAware();
// en CacheWriterTolerante.invoke(...)
try {
    return metodo.invoke(real, args);
} catch (InvocationTargetException envuelta) {
    if (envuelta.getTargetException() instanceof RuntimeException falloDeRedis) {
        log.warn("[cache] Redis fallo en {} {}: {}. Se sigue sin cache.", ...);
        return valorSeguro(metodo.getReturnType());   // null = miss
    }
    throw envuelta.getTargetException();
}
```

El "después". Con la nueva versión y Redis apagado, el GET responde 200 (ahora va a Postgres), la transferencia de 1 peso responde 201 COMPLETADA y los saldos cuadran: Ana bajó de 367 a 366 y Beto subió de 933 a 934. El log dice qué operación de caché falló: se ven fallos de get, put y remove con la clave, por "Unable to connect to Redis" o "Redis command timed out".



Ilustración 36. Con Redis apagado y el arreglo: GET 200, transferencia 201 COMPLETADA y saldos Ana 366 y Beto 934.

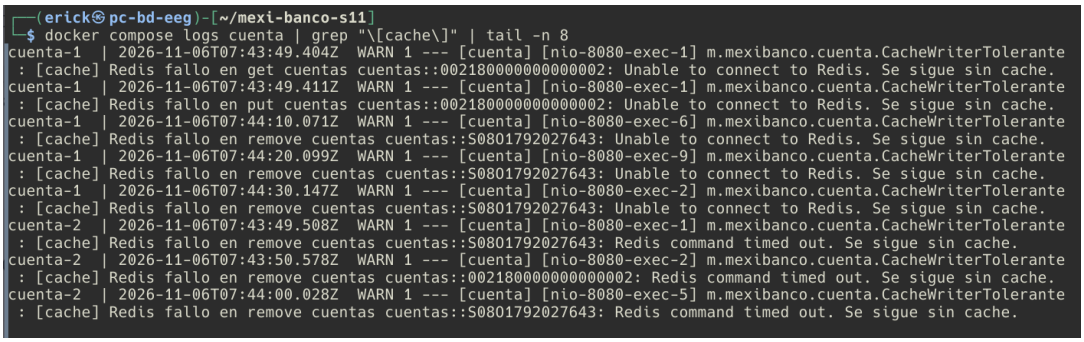


Ilustración 37. Log de cuenta: [cache]

Al encender Redis. Con docker compose start redis todo sigue: el GET trae el saldo actual (366) y la copia vuelve a Redis con un TTL de 30 segundos. Redis arranca vacío porque solo guarda en memoria, así que no queda ningún dato viejo.

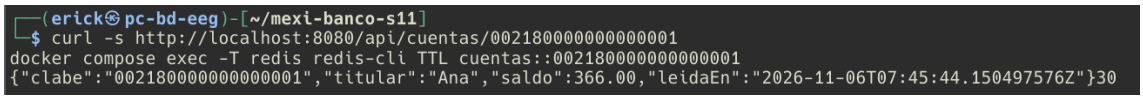


Ilustración 38. Redis encendido otra vez: el GET trae el saldo (366.00) y la copia tiene TTL 30.

Mi conclusión. La caché no debe tumbar nada. Sin Redis el sistema es más lento porque todas las consultas van a Postgres, pero sigue funcionando, y el dinero nunca depende de la caché.

8. Preguntas de cierre

1. **¿Por qué MongoDB para los avisos y no para el saldo?** Porque cada aviso tiene la forma de su canal (un SMS lleva teléfono y texto, un correo lleva correo, asunto y cuerpo, un push lleva token), y en una tabla tendría muchas columnas vacías o una tabla por canal. Un aviso se escribe una vez, no se actualiza y se lee completo por CLABE, que es justo lo que hace bien un documento con su índice. El saldo es dinero, necesita transacciones, que dos operaciones no se pisen y que nunca se contradiga ni se pierda. Eso lo garantiza Postgres con su transacción local y @Version, así que el dinero no cambia de motor.

2. **¿Qué hit ratio mediste y qué compraste con la caché?** En el escenario A medí 99 % (99 hits y 1 miss) y en el B 84 % (101 hits y 19 misses). Baja en B porque cada transferencia borra la copia de Ana y la de Beto, y la siguiente lectura de cada uno es un miss. En la latencia la caché ayudó poco: mediana de 6.2 y 7.3 ms contra 10.4 ms sin ella. Lo que sí compré fue capacidad: con caché Postgres tuvo 3 y 2 lecturas por cada 100 consultas, y sin caché tuvo 100. Compré capacidad más que velocidad.

3. **¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?** Podría aprobar un cargo con un saldo que ya no es real. En la prueba del dato viejo la caché decía que Ana tenía 367 pero en Postgres tenía 5. Si el cargo hubiera leído la caché, habría aprobado la transferencia de 100 y Ana habría quedado con un saldo negativo, o sea dinero que no existe. Como el cargo lee Postgres, respondió 422. La caché puede mostrar un dato viejo, pero nunca decidir.

4. **¿Por qué DEL y no SET, y por qué después del commit?** Con DEL la copia simplemente desaparece y la siguiente consulta la llena con el saldo que hay en Postgres, así que Postgres sigue siendo la única verdad. Con SET tendría que escribir el mismo saldo en dos lugares, y si dos operaciones llegan casi juntas podría dejar en Redis un saldo más viejo que el de la base. Y va después del commit porque, si borrara antes, otra consulta podría leer el saldo viejo de Postgres (el cambio todavía no está confirmado) y volver a guardarlo en la caché, que quedaría con el dato viejo hasta que caduque.